COSC 4370 - Homework 2

Raphael Tinoco 1668116

October 3, 2023

1 Assignment Problems

This homework assignment consisted of two parts. The first part is comprised of three unrelated problems in which I was to recreate 3-D scenes utilizing OpenGL. The second part involved me creating a scene of my imagination, with a couple of constraints and requirements.

Part One: 3-D Scene Recreation

The three scenes I had to recreate are as follows:

1. A set of 8 teapots arranged in a rhombus shape all equidistant from the origin
2. A pyramid shape of 11 steps
3. An upside-down triangle shape of teapots

These three scenes were constructed using a combination of certain OpenGL/freeglut functions. Namely, I utilized a few transformational mechanisms [glPushMatrix(), glPopMatrix(), glTranslatef(x, y, z), glRotate(angle, x, y, z)] as well as some already created objects [glutSolidTeapot(size), glutSolidCube(size)].

Part Two: Creative Scene

This part of the homework was to create a scene of my own imagination, with the requirement of having "at least one instance of nested applications of glPushMatrix()". My scene attempts to create an image of a mouse. One of the required nested applications creates a mouses ears, and another creates its eyes.

The details of how each scene was successfully reconstructed, as well as how I created my own scene are listed below in section 3.

2 Methods and Implementations

Having been given some filled in code for "main.cpp", the instructions were to fill out four different methods: problem1(), problem2(), problem3(), and problem4().

Problem1()

As the goal was to get a teapot on every point of the compass (i.e. N, NE, E, etc.), I first began with getting a teapot to the North point. The way this was done was by doing a glPushMatrix() function call (to have all transformations done only to this "North" matrix), followed by a simple glTranslatef(0, 1, 0) function call to move the matrix one unit north in the y axis. Then, to match the target image, I had to do

two rotations using the glRotatef() function to turn the teapot 180 degrees in both the x and y axis. Finally, I placed the teapot using glutSolidTeapot(.125) [I played with the sizing parameter until I found a value that matched nicely] and did a glPopMatrix() to remove the matrix from the top of the stack.

This process was repeated for all cardinal directions (except those only involved one rotation), and from there I simply did the same process for the intermediate directions (found halfway between two cardinal directions, so translations and rotations were trivial).

Problem2()

This time, I was working with the glutSolidCube() object to create a pyramid shape. Examining the target image, I noticed that the height of each step remained constant, and the x and z values kept getting scaled up by some constant factor.

I first began with getting the top step to be at its correct height to be similarly placed as in the target image, with initial coordinates at (0, 1.5, 1). Again, these values were obtained to match the target image by plugging in different values and choosing what seemed best. From there, I arrived to have the scaling for x and z to be 1.75 for each subsequent step.

Instead of having 11 different matrix pushes / pops like my implementation of problem 1, I utilized a for loop (11 iterations) and simply updated a variable yPos by -0.125 (so next placed step is lower) and another variable xzScale by 1.75 (so next placed step is scaled in the x and z directions). Then I called glTranslatef(xPos, yPos, zPos) where xPos and zPos remain constant at original coordinates, and I called glScalef(xzScale, yScale, xzScale) where yScale remained constant with a value of 1. Following this, I called glutSolidCube(0.125), did a glPopMatrix() and the loop restarted.

Problem3()

This problem involved recreating an upside-down triangle of teapots all pointing the same direction with no rotation. This was implemented similarly to problem2(), but I used a nested for loop that would iterate through each column for a given row of teapots.

The outer loop would loop 6 times for each row i, and the inner loop would loop for i + 1 times (to account for the index starting at 0) for each teapot in the row.

A quick examination of the target image revealed the fact that the row number equaled the number of teapots in the row (when counting from bottom to top). From there, I noticed how even indexed rows (0, 2, and 4) all had teapots at x = 0, while odd indexed rows were offset by a bit. From this realization, I initialized x to be 0 at the start of a row and would be updated by that offset times the row number. In this case, the offset was (-0.3 * i), where i is the row number. There's no real importance to the offset's

magnitude other than that's the number I felt made the image match the target image best. Similarly, I found a good vertical offset between the rows to be y = 0.4.

The inner loop would then push a matrix, translate it according to (x, y, 0), place a teapot, pop the matrix, and increase the value of x by 0.6 for the next teapot in the row. This would continue for each teapot in the row, and for each row in the triangle until all were placed.

Problem4()

This final problem is an image of my own creation. Initially, I was aiming to create an articulated hand as mentioned in the homework as a suggestion. After a bit however, I noticed that my hand was starting to look more like a mouse's face, so I went from there.

I began by using OpenGL immediate mode to create a triangle that I believe was initially either isosceles or equilateral but modified later to better match with edges of other shapes used. I also decided it would be interesting to have the triangle be shaded brown to more resemble a mouse, and used the display() function as a kind of reference on how to do this (as this was how the different axis were colored).

Then I went on to create the mouse's features (eyes, ears, and nose). Again, to avoid having copies of essentially the same transformational mechanisms, I used a loop so I only had to create one ear and transform it on the x axis a bit. Similarly, I only created one eye and translated it a bit. The loop is as follows: indexes 0 and 4 relate to the ears, and indexes 1 and 2 relate to the eyes. Index 5 is the nose.

Ears:

The ears were to be placed on the edge of the triangle's back two high vertexes. Left vertex (-1, 0, -2); right vertex (2.5, 0, -2). Thus, I initialized a variable x to -2 outside of the loop and incremented x by 1 for every iteration in the loop. This way, each different facial feature will be equally spaced out. After finding the perfect placement for the left ear with much trial and error, I did a glPushMatrix() followed by a glTranslatef() call with some doubles as parameters, utilizing x + 0.25 for x. Then, I scaled the matrix with glScalef(1, 1.5, 1) to increase the size of the cube in the y direction. Finally I placed the cube with glutSolidCube().

At this point, I did not do a glPopMatrix() as I had to satisfy the problem requirement. So, I did a glPushMatrix() to build on the previous transformational mechanisms. As the original matrix had no rotation to it, I knew I wanted to incorporate that here, and then also once more for the next nested matrix. (Remember, at this point, I was creating a finger and had envisioned a finger curling slightly). So, I did another scale, with a slightly smaller y scaling. Then I did a glTranslatef(0, 0.3, 0.1) to move the matrix up and not be hidden by the previous cube. Finally, I did a glRotatef() and placed the cube with glutSolidCube(). This was repeated in the final nesting of the top third of the "ear", and after placing that cube, I did 3 glPopMatrix() operations.

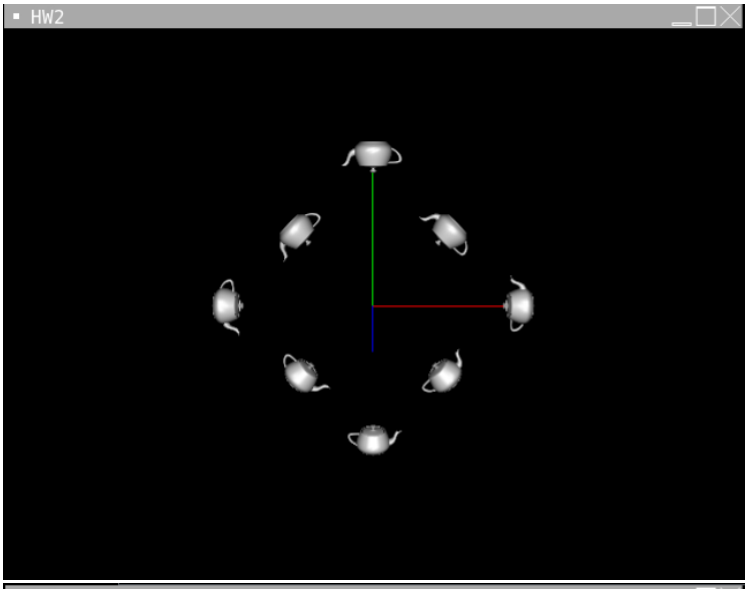
Eyes

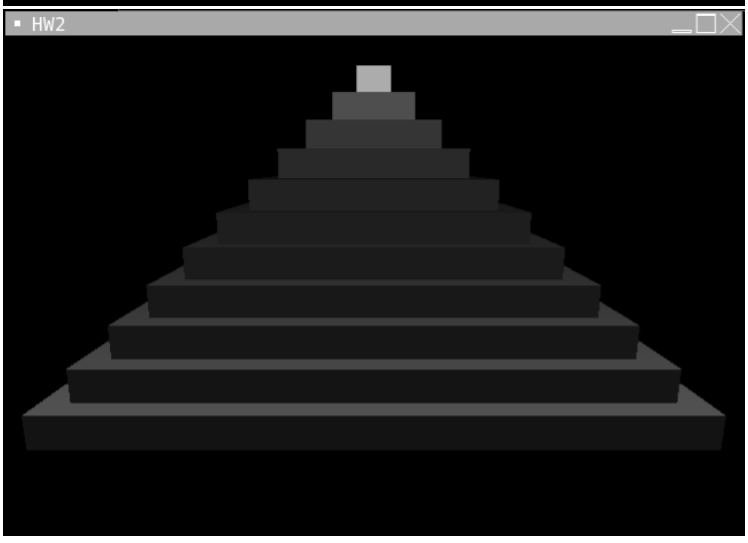
The process for creating the eyes is very similar as to the ears. There is a 3 step process: placement of initial cube push that matrix, then scale up, translate, rotate, place and then push that matrix, and finally another rotate, scale, translate, place and 3 pops.

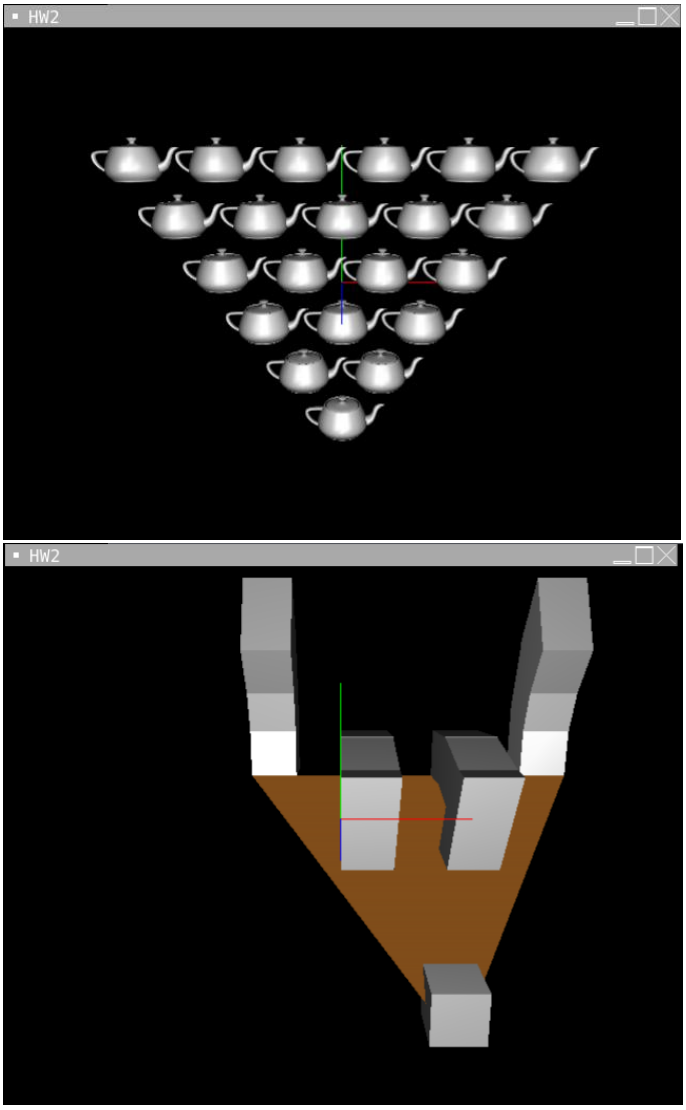
Nose

The placement of the nose was very trivial, as I centered the bottom vertex to be in a cubes center. The cube was created by a matrix push, translate, place, and pop.

3 Results







4 Other Materials

Links used to work on project:

1. https://freeglut.sourceforge.net/docs/api.php#GeometricObject
2. https://math.hws.edu/bridgeman/courses/324/s06/doc/opengl.html